DATA ANALYTICS

Data Cleaning

Theory, Principles & Python Implementation

By MICHAEL JOHN S. RIVERA, MBAn

Topics: Missing Values · Duplicates · Format Errors · Outliers · Incorrect Values · Standardization

Data Cleaning in the Analytics Life Cycle

Data cleaning does not happen in isolation — it is informed by EDA findings and business policy constraints.

Your Blueprint — What You Already Know

A quick recap before we execute. Every fix today is already justified.

You found the problems. The policies gave you the rules. Today we execute.

Field	EDA Found	Business Rule	Action
gpa	`518 nulls + 81 'N/A' strings`	Nullable for 1st-sem only	Replace N/A→NaN, fill median
last_name	`~120 null rows`	Required — cannot be empty	Drop rows where null
student_id	`400 duplicate IDs`	Must be unique per student	drop_duplicates keep=first
age	`~300 outliers (0, 999, negatives)`	Valid range: 16-26 (undergrads)	Replace out-of-range with NaN
gender	`21 unique variants`	Only M / F accepted	Map all variants to M or F
contact_no	`~145 dashes, ~150 wrong length`	11 digits, no dashes	Strip dashes, flag wrong len
department	`~200 inconsistent codes`	10 approved dept codes only	Map variants to standard code

This table is your cleaning contract. Every fix today traces back to your EDA report and business policy decisions.

The Chain — Every Fix Must Be Justified

EDA Finding → Business Rule → Cleaning Code | No chain = no action

Every single cleaning action today follows this chain. If a fix cannot be traced back to both, we do not do it.

EDA FOUND

gpa: 518 nulls
+ 81 'N/A' strings
dtype = object
(should be float)

→

BUSINESS RULE SAYS

GPA nullable for
1st-semester students.
'N/A' is not a valid null.
Must store as NaN.

→

CLEANING CODE

```
df.replace('N/A',np.nan)
df['gpa'].fillna(
   df['gpa'].median(),
   inplace=True)
```

Another example — gender:

EDA FOUND

gender: 21 unique values
'male','MALE','m',
'Female','female'...

→

BUSINESS RULE SAYS

Enrollment form uses
only 'M' and 'F'.
No other values allowed.

→

CLEANING CODE

```
gmap={'male':'M','MALE':'M',
'female':'F','Female':'F'}
df['gender'].replace(gmap)
```

No EDA finding + no business rule = no cleaning action. This is the discipline that separates professional cleaning from guesswork.

What We Fix Today — 6 Cleaning Types on Real Data

All decisions already justified. Blueprint confirmed. Let's execute.

Each part below traces to a specific EDA finding and a specific business rule. No new decisions needed.

01

Missing Values

Fields: gpa, last_name, email

EDA: 518 null gpa + 81 N/A strings; ~120 null last_name

Rule: GPA nullable 1st sem. last_name required.

02

Duplicate Records

Fields: student_id

EDA: 400 duplicate IDs found in EDA

Rule: One record per student. student_id unique.

03

Format & Structural

Fields: names, gender, status

EDA: ~160 ALL-CAPS names; ~180 gender variants; whitespace

Rule: Title Case. M/F only. Standard status codes.

04

Outlier Detection

Fields: age, gpa

EDA: ~300 age outliers; ~330 GPA outside 1.00-3.50

Rule: Age 16-26. GPA 1.00-3.50 (PH scale).

05

Incorrect Values

Fields: year_level, email, names

EDA: ~180 invalid yr_lvl; ~150 bad gender; ~100 placeholders

Rule: yr_level 1-5. Institutional email only.

06

Standardization

Fields: contact_no, department

EDA: ~145 dashes; ~150 wrong length; ~200 inconsistent depts

Rule: contact_no 11 digits. 10 approved dept codes.

Blueprint confirmed. Open Jupyter. Load enrollment_dirty_10k.csv. Let us clean.

What We Will Cover

PART 1
Missing Values

PART 2
Duplicate Records

PART 3
Format & Structural Errors

PART 4
Outlier Detection

PART 5
Incorrect Values

PART 6
Standardization & Validation

Each part: Theory (definition + types + when to apply) → Python/pandas implementation → Before & After examples

🛑 STOP

HANDS-ON — PART 1: MISSING VALUES

📌 This dataset has 860 missing values across 5 columns — some as empty cells, some as the string 'N/A'. Write pandas code to: (a) count nulls per column, (b) detect 'N/A' strings in gpa, (c) drop rows where last_name is empty, (d) fill missing gpa with the column median.

student_id	last_name	first_name	age	yr_lvl	gpa	status
202408933	San Pedro	Clifford	22	1	(empty)	Enrolled
202408998	Obando	Ryan	28	1	(empty)	Enrolled
202403897	Natividad	Iris	17	1	N/A	Enrolled
202401999	Reyes	Rowena	26	1	N/A	LOA
202401258	(empty)	Bryson	22	4	1.54	LOA
202408911	(empty)	Grace	26	3	1.57	LOA

TRIGGERED BY EDA: 518 null gpa + 81 N/A strings; ~120 null last_name; ~160 null email **RULE:** GPA nullable for 1st-semester students. N/A is not a valid null. last_name is required.

✅ SOLUTION

PART 1: MISSING VALUES — Detect · Replace N/A string · Drop nulls · Impute median

```
import pandas as pd
import numpy as np

df = pd.read_csv('students_dirty_10k.csv')

# (a) Count nulls — blank cells load as NaN
print(df.isnull().sum())
# gpa: ~518   email: ~160
# last_name: ~120  contact_no: ~120

# (b) Detect 'N/A' string (NOT caught by isnull)
print(df[df['gpa']=='N/A'].shape)  # ~81 rows

# Replace 'N/A' string with actual NaN
df.replace('N/A', np.nan, inplace=True)

# (c) Drop rows where last_name is null
```

After cleaning:

student_id	last_name	yr	action	gpa
202408933	San Pedro	1	→ gpa filled	2.43
202408998	Obando	1	→ gpa filled	2.43
202403897	Natividad	1	N/A → NaN → filled	2.43
202401258	(empty last_name)	4	→ row DROPPED	—
202408911	(empty last_name)	3	→ row DROPPED	—

📌 What to observe: Check df.shape before and after. Always validate with df.isnull().sum() and df.describe() to confirm the fix worked as expected.

```
df['gpa'].fillna(median_gpa,
```

Missing Values — Python (pandas)

DETECT

```python
# Count missing per column
df.isnull().sum()

# Percentage missing
df.isnull().mean() * 100

# Rows with ANY missing value
df[df.isnull().any(axis=1)]

# Missing in a specific column
df[df['email'].isnull()]
```

HANDLE

```python
# Drop rows (required field)
df.dropna(subset=['last_name'])

# Fill with constant
df['rating'].fillna(0)

# Fill with mean / median
df['age'].fillna(df['age'].mean())
df['salary'].fillna(df['salary'].median())

# Forward fill (time-series)
df['score'].ffill()
```

	BEFORE	AFTER
email	NaN	flagged / dropped per rule
rating	NaN	0 (default per policy)

🛑 STOP

HANDS-ON — PART 2: DUPLICATE RECORDS

📌 The dataset has ~200 duplicate student_id values — some exact copies, some with a different email. Write pandas code to: (a) find all rows with a duplicated student_id, (b) count total duplicate rows, (c) remove duplicates keeping the first occurrence.

student_id	last_name	first_name	email
202400534	Delos Reyes	Maricel	alt_mariceldelosreyes4@uni.edu.ph
202400534	Delos Reyes	Maricel	mariceldelosreyes4@uni.edu.ph
202401843	Jimenez	Carlos	carlosjimenez75@uni.edu.ph
202401843	Jimenez	Carlos	carlosjimenez75@uni.edu.ph
202401813	Castillo	Rafael	rafaelcastillo93@uni.edu.ph
202401813	Castillo	Rafael	alt_rafaelcastillo93@uni.edu.ph

TRIGGERED BY EDA: 400 rows with duplicate student_id — some exact copies, some with different email **RULE:** student_id must be unique. One record per student per semester. Keep first occurrence.

✅ SOLUTION

PART 2: DUPLICATE RECORDS — Detect · Count · Remove keeping first

```
# (a) Find all rows with duplicated student_id
dupes = df['student_id'].duplicated(
    keep=False
)
print(df[dupes][['student_id','last_name',
                 'email']].head(10))
# ~400 rows flagged

# (b) Count exact full duplicates
print("Full dupes:",
      df.duplicated().sum())     # ~150
print("ID dupes:",
      df['student_id']
      .duplicated(keep=False)
      .sum())                    # ~400

# (c) Drop — keep first occurrence
```

After cleaning:

student_id	last_name	first_name	result
202400534	Delos Reyes	Maricel	kept (first)
202400534	(duplicate)	Maricel	dropped ✓
202401843	Jimenez	Carlos	kept (first)
202401843	(duplicate)	Carlos	dropped ✓
202401813	Castillo	Rafael	kept (first)

📌 What to observe: Check df.shape before and after. Always validate with df.isnull().sum() and df.describe() to confirm the fix worked as expected.

Duplicate Records — Python (pandas)

```python
# Check for full duplicates
df.duplicated().sum()

# View all duplicate rows
df[df.duplicated(keep=False)]

# Duplicates by specific column
df[df['student_id'].duplicated(
    keep=False
)]

# Drop: keep first occurrence
df.drop_duplicates(
    subset=['student_id'],
    keep='first',
    inplace=True
)

# Drop: keep last occurrence
df.drop_duplicates(
    subset=['student_id'],
```

```python
# Keep row with fewest nulls
# (most complete record)
df['null_count'] = df.isnull().sum(
    axis=1
)
df_sorted = df.sort_values(
    ['student_id','null_count']
)
df_clean = df_sorted.drop_duplicates(
    subset=['student_id'],
    keep='first'
).drop(columns='null_count')

# Reset index after dropping
df_clean = df_clean.reset_index(
    drop=True
)
```

⚠️ Business Rule Reminder: Who decides which duplicate to keep — first, last, or most complete? Always confirm with the domain expert before applying .drop_duplicates().

🛑 STOP

HANDS-ON — PART 3: FORMAT & STRUCTURAL ERRORS

📌 The dataset has ~160 ALL-CAPS last names, ~180 gender variants, ~130 names with whitespace, and ~140 enrollment_status casing errors. Write pandas code to: (a) strip whitespace from first_name, (b) standardize gender, (c) fix enrollment_status casing.

student_id	last_name	first_name	gender	status
202403710	GARCIA	Leo	M	Enrolled
202400198	VILLANUEVA	Nina	F	Enrolled
202403626	Cabrera	Luisa	FEMALE	Enrolled
202409450	Torres	Diana	female	Enrolled
202405223	Alcantara	Pedro	M	LOA
202400626	Fajardo	Patrick	M	ENROLLED

TRIGGERED BY EDA: ~160 ALL-CAPS last names; ~180 gender variants; ~130 whitespace names **RULE:** Names in Title Case per data entry standard. Gender must be M or F only.

✅ SOLUTION

PART 3: FORMAT & STRUCTURAL ERRORS — Whitespace · Gender casing · Status casing · Name casing

```
# (a) Strip whitespace from first_name
df['first_name'] = (
    df['first_name'].str.strip()
)
# ~130 rows fixed

# (b) Standardize gender
gender_map = {
    'male':'M',   'Male':'M',
    'MALE':'M',   'm':'M',
    'female':'F', 'Female':'F',
    'FEMALE':'F', 'f':'F'
}
df['gender'].replace(
    gender_map, inplace=True
)
# ~180 rows fixed
```

After cleaning:

student_id	last_name / gender	gender	result
202403710	GARCIA → Garcia	M	fixed ✓
202403626	FEMALE → F	F	fixed ✓
202405223	Pedro → Pedro	M	fixed ✓
202400626	ENROLLED → Enrolled	M	fixed ✓
202409450	female → F	F	fixed ✓

📌 What to observe: Check df.shape before and after. Always validate with df.isnull().sum() and df.describe() to confirm the fix worked as expected.

Format & Structural Errors — Python (pandas)

```python
# ── CASING ──────────────────────
# Lowercase all text in a column
df['gender'] = df['gender'].str.lower()

# Uppercase
df['status'] = df['status'].str.upper()

# Title case (for names)
df['last_name'] = (
    df['last_name'].str.title()
)

# ── WHITESPACE ──────────────────
df['name'] = df['name'].str.strip()

# ── SPECIAL CHARACTERS ──────────
df['name'] = df['name'].str.replace(
    r'[^a-zA-Z\s]', '', regex=True
)
```

```python
# ── DATA TYPES ──────────────────
# Check current types
df.dtypes

# Convert string to integer
df['age'] = df['age'].astype(int)

# Convert to float
df['salary'] = df['salary'].astype(
    float
)

# ── DATE FORMAT ─────────────────
df['date'] = pd.to_datetime(
    df['date'],
    format='%m/%d/%Y',
    errors='coerce'  # NaT if fails
)

# ── REPLACE PLACEHOLDERS ────────
df.replace('N/A', pd.NA, inplace=True)
df.replace('', pd.NA, inplace=True)
```

Tip: Always run df.dtypes and df.head() after type conversions to confirm the change was applied correctly.

🛑 STOP

HANDS-ON — PART 4: OUTLIER DETECTION

📌 The dataset has ~300 age outliers (extreme, negative, zero) and ~330 gpa outliers (above 5.00 or below 1.00). Write pandas code to: (a) use IQR on age to detect outliers, (b) apply domain rule for gpa (valid: 1.00–3.50), (c) remove both groups.

student_id	last_name	first_name	age	gpa	dept
202409191	Mendoza	Renz	510	2.62	BSIT
202404626	De Leon	Karl	959	3.02	BSCS
202407003	Lacson	Carlos	-4	2.21	BSN
202401917	Castillo	Dominic	-23	2.12	BSECE
202404455	Bonifacio	Ana	20	5.81	BSBA
202402604	Macaraeg	Clara	23	0.52	BSED

TRIGGERED BY EDA: ~300 age outliers (0, 999, negative); ~330 GPA values outside 1.00-3.50 **RULE:** Age: 16-26 per enrollment policy. GPA: 1.00-3.50 per Philippine grading scale.

✅ SOLUTION

PART 4: OUTLIER DETECTION — IQR on age · Domain rule on gpa · Remove both

```
import pandas as pd

df = pd.read_csv('students_dirty_10k.csv')
df['age'] = pd.to_numeric(df['age'],
                          errors='coerce')
df['gpa'] = pd.to_numeric(df['gpa'],
                          errors='coerce')

# (a) IQR method on age
Q1 = df['age'].quantile(0.25)
Q3 = df['age'].quantile(0.75)
IQR = Q3 - Q1
bad_age = (
    (df['age'] < Q1 - 1.5*IQR) |
    (df['age'] > Q3 + 1.5*IQR) |
    (df['age'] <= 0)
)
```

After cleaning:

student_id	value	result	reason
202409191	510	→ removed	age outlier
202404626	959	→ removed	age outlier
202407003	-4	→ removed	age negative
202404455	5.81 gpa	→ removed	gpa > 3.50
202402604	0.52 gpa	→ removed	gpa < 1.00

📌 What to observe: Check df.shape before and after. Always validate with df.isnull().sum() and df.describe() to confirm the fix worked as expected.

Outlier Detection — Python (pandas)

```python
# ── Z-SCORE METHOD ──────────────
from scipy import stats
import numpy as np

z_scores = np.abs(
    stats.zscore(df['salary'].dropna())
)
# Flag rows where z > 3
outliers_z = df[z_scores > 3]
print(f"Outliers: {len(outliers_z)}")

# ── IQR METHOD ──────────────────
Q1 = df['salary'].quantile(0.25)
Q3 = df['salary'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers_iqr = df[
    (df['salary'] < lower) |
    (df['salary'] > upper)
]
```

```python
# ── DOMAIN THRESHOLD ────────────
# Based on business rule:
# salary must be between 500-200000
invalid_sal = (
    (df['salary'] < 500) |
    (df['salary'] > 200000)
)
df[invalid_sal]

# ── TREATMENT: WINSORIZE ────────
# Cap at upper/lower bounds
df['salary'] = df['salary'].clip(
    lower=lower, upper=upper
)

# ── TREATMENT: REMOVE ───────────
df_clean = df[~invalid_sal].copy()

# ── QUICK VISUAL CHECK ──────────
df.boxplot(column=['salary'])
df['salary'].describe()
```

Always plot before and after: df['salary'].hist() or df.boxplot() to confirm the treatment had the intended effect.

🛑 STOP

HANDS-ON — PART 5: INCORRECT VALUES

📌 The dataset has ~180 impossible year_level values, ~150 invalid gender categories, ~220 wrong-domain emails, and ~100 test/placeholder records. Write pandas code to detect and flag each group — do NOT auto-remove without checking.

student_id	last_name	first_name	yr_lvl	gender	email
202408272	Del Rosario	Francis	7	M	francisdelrosario81@uni.edu.ph
202402693	Hidalgo	Ramon	6	M	ramonhidalgo68@uni.edu.ph
202403190	Hidalgo	Nina	5	none	ninahidalgo98@uni.edu.ph
202408663	Ilustre	Adrian	1	unknown	adrianilustre59@uni.edu.ph
202401872	Ramos	Mark	3	M	mark48@protonmail.com
202405027	sample	sample	3	M	sample@uni.edu.ph

TRIGGERED BY EDA: ~180 impossible year_level values; ~150 invalid gender codes; ~100 placeholder records **RULE:** year_level: 1-5 only. Gender: M or F only. Institutional email only. No dummy records.

✅ SOLUTION

PART 5: INCORRECT VALUES — Year level · Invalid gender · Wrong email domain · Placeholders

```
valid_gender = ['M','F','Prefer not to say']
test_names   = ['test','xxx','sample','n/a',
                'unknown','asdf','temp','dummy']

# (a) Invalid year_level
bad_yr = ~df['year_level'].astype(str).apply(
    lambda x: x.isdigit() and
              1 <= int(x) <= 5
)
print("Bad year_level:", bad_yr.sum())
# Expected: ~180

# (b) Invalid gender category
bad_gender = ~df['gender'].isin(
    valid_gender
)
print("Bad gender:", bad_gender.sum())
```

After cleaning:

student_id	issue	result	reason
202408272	yr=7	→ removed	year_level invalid
202403190	gender=none	→ removed	invalid category
202401872	protonmail	→ flagged	email_flag=True
202405027	sample/sample	→ removed	placeholder record
202408663	gender=unknown	→ removed	invalid category

📌 What to observe: Check df.shape before and after. Always validate with df.isnull().sum() and df.describe() to confirm the fix worked as expected.

Incorrect Values — Python (pandas)

```python
# ── RULE VIOLATION ──────────────
# year_level must be 1–5
invalid_yr = ~df['year_level'].between(
    1, 5
)
df[invalid_yr]

# ── INVALID CATEGORY ────────────
valid_gender = ['M', 'F',
                'Prefer not to say']
invalid_g = ~df['gender'].isin(
    valid_gender
)
df.loc[invalid_g, 'gender'] = pd.NA

# ── LOGICAL INCONSISTENCY ───────
# Grad year cannot be before enroll
illogical = (
    df['grad_year'] <
    df['enrollment_year']
)
```

```python
# ── PLACEHOLDER DETECTION ───────
# Flag suspicious name entries
test_names = ['test','unknown',
    'n/a','xxx','sample']
suspect = df['last_name'].str.lower(
).isin(test_names)
df[suspect]

# ── CROSS-COLUMN VALIDATION ─────
# Age vs year level consistency
# (year 1 student aged 60 is odd)
flag = (
    (df['year_level'] == 1) &
    (df['age'] > 50)
)
df[flag]  # Review — don't auto-drop

# ── CORRECT KNOWN ERRORS ────────
# Map common misspellings
df['gender'].replace({
    'male':'M', 'female':'F'
```

Key Principle: Incorrect values require the most judgment. When in doubt, FLAG the record — do not silently drop or correct without business validation.

🛑 STOP

HANDS-ON — PART 6: STANDARDIZATION

📌 The dataset has ~145 contact numbers with dashes, ~150 with wrong digit count, ~200 inconsistent department codes, and ~100 stripped student IDs. Write pandas code to: (a) remove dashes from contact_no, (b) flag wrong-length contacts, (c) standardize department codes.

student_id	last_name	contact_no	department
202401872	Ramos	0983-234-4261	BSIT
202404328	Macaraeg	0946-994-9756	BSIT
202408375	Hernandez	09396124	BSCS
202409005	Fernandez	097731334	BSCS
202404590	Villanueva	09975081753	B.S.B.A
202405223	Alcantara	09984576710	bsit

TRIGGERED BY EDA: ~145 contact_no with dashes; ~150 wrong digit count; ~200 inconsistent department codes **RULE:** contact_no: 11 digits, no dashes. Department: one of 10 approved codes only.

✅ SOLUTION

PART 6: STANDARDIZATION — Contact format · Department codes · Name casing

```
valid_depts = ['BSIT','BSCS','BSECE','BSME',
    'BSCE','BSN','BSBA','BSED',
    'BSPSYCH','BSCRIM']

# (a) Remove dashes from contact_no
df['contact_no'] = (
    df['contact_no']
    .str.replace('-','', regex=False)
)
# ~145 dashes removed

# (b) Flag wrong-length contacts
bad_contact = (
    df['contact_no'].notna() &
    (df['contact_no'].str.len() != 11)
)
print("Wrong length:", bad_contact.sum())
```

After cleaning:

student_id	original	result	status
202401872	0983-234-4261	→ 09832344261	fixed ✓
202408375	09396124	→ flagged	wrong length
202404590	B.S.B.A	→ BSBA	fixed ✓
202405223	bsit	→ BSIT	fixed ✓
202409005	097731334	→ flagged	wrong length

📌 What to observe: Check df.shape before and after. Always validate with df.isnull().sum() and df.describe() to confirm the fix worked as expected.

Post-Cleaning Validation & Documentation

Cleaning is not complete until you verify the output AND record what you did.

✅ Validation Checklist

```
# Shape: rows removed?
print(df_clean.shape)

# Missing values resolved?
df_clean.isnull().sum()

# No duplicates remain?
df_clean.duplicated().sum()

# Data types correct?
df_clean.dtypes

# Value ranges valid?
df_clean.describe()

# Spot-check sample rows
```

📝 Cleaning Log (Required)

Original row count	**1,245 rows**
Rows dropped (nulls)	**18 rows**
Rows dropped (dupes)	**7 rows**
Values corrected	**34 values**
Values imputed	**22 values**
Final row count	**1,220 rows**
Date cleaned	**2025-01-15**

Why document? In professional practice, every cleaning decision must be traceable, reproducible, and justifiable. A cleaned dataset without a cleaning log is unverifiable and unpublishable.

The Complete Cleaning Pipeline — Python Template

```python
import pandas as pd
import numpy as np

# 1. LOAD & PROFILE
df = pd.read_csv('students_dirty_10k.csv')
print("Shape:", df.shape)
print(df.dtypes)
print(df.isnull().sum())

# 2. REPLACE STRING PLACEHOLDERS
df.replace('N/A', np.nan, inplace=True)

# 3. MISSING VALUES
df.dropna(subset=['last_name',
                  'first_name'],
          inplace=True)
df['gpa'].fillna(
    df['gpa'].median(),
    inplace=True
)

# 4. DUPLICATES
df.drop_duplicates(
    subset=['student_id'],
    keep='first',
    inplace=True
)
```

```python
# 5. FORMAT & TYPE FIXES
df['first_name'] = (
    df['first_name'].str.strip()
                    .str.title()
)
df['last_name'] = (
    df['last_name'].str.title()
)
df['gender'].replace(
    {'male':'M','Male':'M','MALE':'M',
     'm':'M','female':'F','Female':'F',
     'FEMALE':'F','f':'F'}, inplace=True
)
df['contact_no'] = (
    df['contact_no']
    .str.replace('-','',regex=False)
)

# 6. CONVERT NUMERIC COLUMNS
df['age'] = pd.to_numeric(
    df['age'], errors='coerce')
df['gpa'] = pd.to_numeric(
    df['gpa'], errors='coerce')

# 7. OUTLIERS & INCORRECT VALUES
df = df[df['age'].between(17,35)]
```

Module Summary

1. Data cleaning is a systematic, rule-governed process — not guesswork. Every decision must be justifiable.
2. The 6 types: Missing Values, Duplicates, Format Errors, Outliers, Incorrect Values, and Standardization each require different detection and treatment strategies.
3. EDA tells you what is wrong. Business policies tell you why it is wrong. Data cleaning is how you fix it.
4. In Python/pandas, each cleaning type translates into Boolean conditions, string methods, type conversions, or statistical bounds.
5. Always validate after cleaning — check shape, nulls, dtypes, and value ranges. A clean dataset without a cleaning log is incomplete.

Next: Lab Exercise — Applying All Six Cleaning Types on a Real Dataset